

Bachelorarbeit:
Experimentelle Analyse von
Algorithmen für fair-k-center mit
Ausreißern

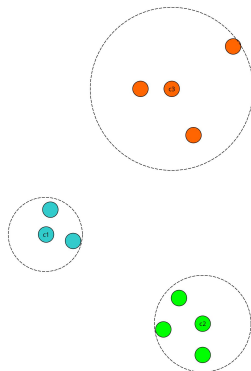
Carsten Krollmann

- 1 Einleitung / Problemstellung
- 2 Algorithmen
- 3 Testdaten
- 4 Experimente
- 5 Auswertung / Fazit

- zur Analyse von Daten
 - unsupervised
 - Nähe von Datenpunkten finden
- Zielfunktion hier k-center
 - maximaler Radius minimieren
 - Zentren aus der Punktemenge
- optimales k-center $\in NP^a$
- Algorithmus von Gonzalez als bekannter Vertreter

^a[Melanie Schmidt](#). „Kombinatorische Algorithmen für Clustering-probleme - Vorlesungsskript“. 2023.

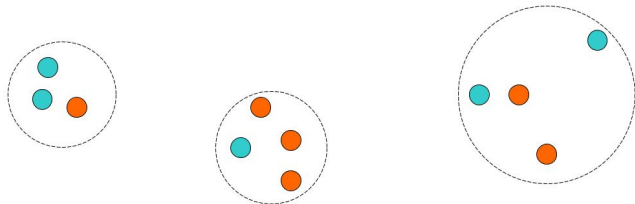
Optimales k-center mit $k = 3$



- faires Attribut
- Zusatzbedingung: gleichmäßig Farben aufteilen auf Cluster
- optimales fair k-center ebenfalls $\in NP^a$

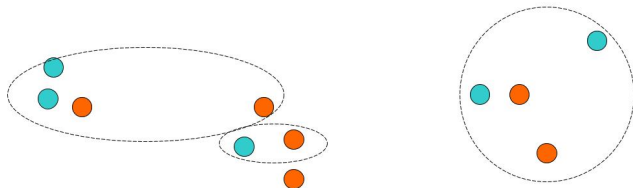
^a[Melanie Schmidt](#). „Kombinatorische Algorithmen für Clustering-probleme - Vorlesungsskript“. 2023.

- faires Attribut
- Zusatzbedingung: gleichmäßig Farben aufteilen auf Cluster
- optimales fair k-center ebenfalls $\in NP^a$



^aMelanie Schmidt. „Kombinatorische Algorithmen für Clustering-probleme - Vorlesungsskript“. 2023.

- faires Attribut
- Zusatzbedingung: gleichmäßig Farben aufteilen auf Cluster
- optimales fair k-center ebenfalls $\in NP^a$



^aMelanie Schmidt. „Kombinatorische Algorithmen für Clustering-probleme - Vorlesungsskript“. 2023.

- Vergleich 2 verschiedener Ansätze
 - Red-Clustering^a
 - Fast-Anchor-Clustering^b
- Nutzen von Fairlets
 - kleinst mögliche Clustereinheit
- beschränkt auf 2 Farben
- beide Algorithmen verallgemeinbar
- Aufteilung in 1:1 Cluster

^aFlavio Chierichetti u. a. *Fair Clustering Through Fairlets*. 2018. arXiv: 1802.05733 [cs.LG].

^bIrina Fast. „Fair k-Center Clustering mit Ausreißern“. Düsseldorf, NRW, Germany, Dez. 2023.

- Vergleich 2 verschiedener Ansätze
 - Red-Clustering^a
 - Fast-Anchor-Clustering^b
- Nutzen von Fairlets
 - kleinst mögliche Clustereinheit
- beschränkt auf 2 Farben
- beide Algorithmen verallgemeinbar
- Aufteilung in 1:1 Cluster

Zusatz: Ausreißer

- reelle Daten nicht perfekt 1:1
- so viele Ausreißer sodass 1:1
 - werden nicht mehr in Cluster berücksichtigt
- bei $n : n + k$ Verteilung $\rightarrow k$ Ausreißer

^aFlavio Chierichetti u. a. *Fair Clustering Through Fairlets*. 2018. arXiv: 1802.05733 [cs.LG].

^bIrina Fast. „Fair k-Center Clustering mit Ausreißern“. Düsseldorf, NRW, Germany, Dez. 2023.

Was war die Mission?

Was war die Mission?

Implementierung der Algorithmen

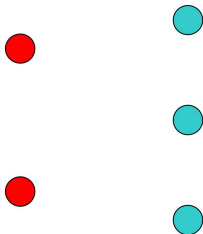
Vergleich durch Testen der Laufzeit
und Güte

- kleinste, faire Clustereinheit
 - hier 1:1 (Rot : Blau)
- Nutzen von Fairlets
 - Fairlets immer zusammen zu Clustern hinzufügen
 - → Cluster immer ausgeglichen
- können optimal berechnet werden über Matching
- Unterschied zwischen Red-Clustering und Fast-Anchor-Clustering

Finden der Fairlets

- 1 bipartiter Graph mit roten und blauen Knoten aufbauen
- 2 mit Quelle s und Senke t zum Flussnetzwerk machen
- 3 Kanten $s \rightarrow$ Rot und Blau $\rightarrow t$ einfügen
- 4 Kanten zwischen den Mengen nach Bedingung:
 - $r_{test} \geq dist(a, b)$ dann Kante
- 5 maximaler Fluss berechnen
- 6 bestes überdeckendes Matching suchen
- 7 gematchte Punkte sind Fairlets
- 8 nicht gematchte Punkte $\rightarrow$ Ausreißer

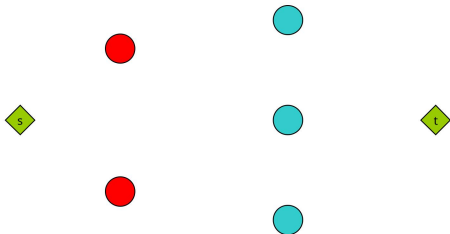
Beispiel für ein Flussnetzwerk



Finden der Fairlets

- 1 bipartiter Graph mit roten und blauen Knoten aufbauen
- 2 mit Quelle s und Senke t zum Flussnetzwerk machen
- 3 Kanten $s \rightarrow$ Rot und Blau $\rightarrow t$ einfügen
- 4 Kanten zwischen den Mengen nach Bedingung:
 - $r_{test} \geq dist(a, b)$ dann Kante
- 5 maximaler Fluss berechnen
- 6 bestes überdeckendes Matching suchen
- 7 gematchte Punkte sind Fairlets
- 8 nicht gematchte Punkte $\rightarrow$ Ausreißer

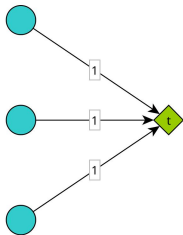
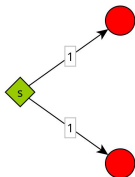
Beispiel für ein Flussnetzwerk



Finden der Fairlets

- 1 bipartiter Graph mit roten und blauen Knoten aufbauen
- 2 mit Quelle s und Senke t zum Flussnetzwerk machen
- 3 Kanten $s \rightarrow \text{Rot}$ und $\text{Blau} \rightarrow t$ einfügen
- 4 Kanten zwischen den Mengen nach Bedingung:
 - $r_{\text{test}} \geq \text{dist}(a, b)$ dann Kante
- 5 maximaler Fluss berechnen
- 6 bestes überdeckendes Matching suchen
- 7 gematchte Punkte sind Fairlets
- 8 nicht gematchte Punkte $\rightarrow$ Ausreißer

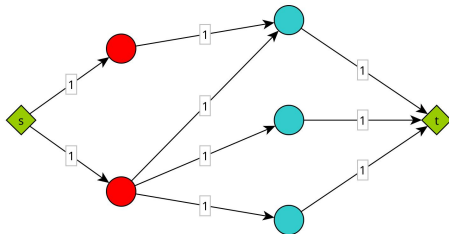
Beispiel für ein Flussnetzwerk



Finden der Fairlets

- 1 bipartiter Graph mit roten und blauen Knoten aufbauen
- 2 mit Quelle s und Senke t zum Flussnetzwerk machen
- 3 Kanten $s \rightarrow \text{Rot}$ und $\text{Blau} \rightarrow t$ einfügen
- 4 Kanten zwischen den Mengen nach Bedingung:
 - $r_{\text{test}} \geq \text{dist}(a, b)$ dann Kante
- 5 maximaler Fluss berechnen
- 6 bestes überdeckendes Matching suchen
- 7 gematchte Punkte sind Fairlets
- 8 nicht gematchte Punkte $\rightarrow$ Ausreißer

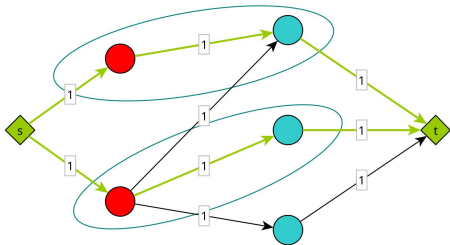
Beispiel für ein Flussnetzwerk



Finden der Fairlets

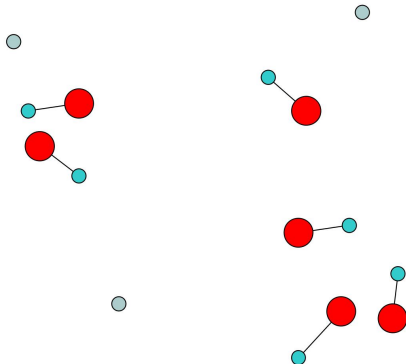
- 1 bipartiter Graph mit roten und blauen Knoten aufbauen
- 2 mit Quelle s und Senke t zum Flussnetzwerk machen
- 3 Kanten $s \rightarrow \text{Rot}$ und $\text{Blau} \rightarrow t$ einfügen
- 4 Kanten zwischen den Mengen nach Bedingung:
 - $r_{\text{test}} \geq \text{dist}(a, b)$ dann Kante
- 5 maximaler Fluss berechnen
- 6 bestes überdeckendes Matching suchen
- 7 gematchte Punkte sind Fairlets
- 8 nicht gematchte Punkte $\rightarrow$ Ausreißer

Beispiel für ein Flussnetzwerk
 $|f| = 2$



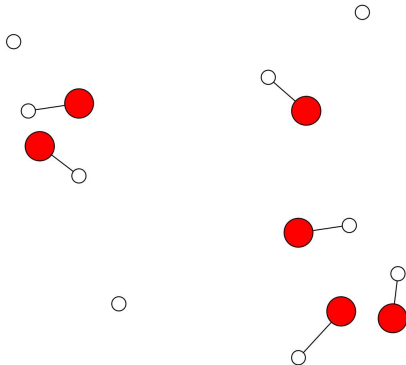
Clustern mit Fairlets

- 1 nur rote Punkte clustern
 - mit Gonzalez
- 2 alle blauen Punkte zu ihren Partnern zuweisen
- 3 Ausreißer bleiben bestehen



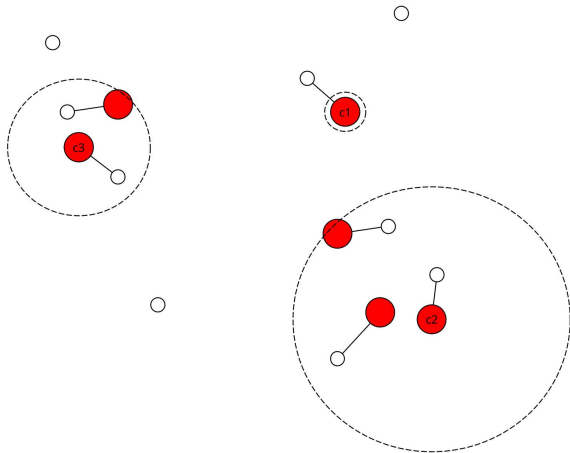
Clustern mit Fairlets

- 1 nur rote Punkte clustern
 - mit Gonzalez
- 2 alle blauen Punkte zu ihren Partnern zuweisen
- 3 Ausreißer bleiben bestehen



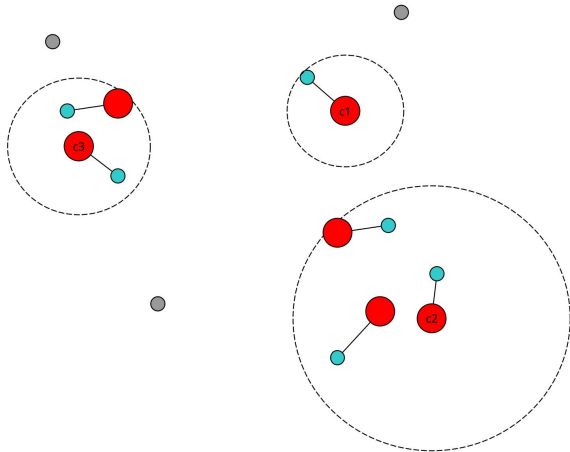
Clustern mit Fairlets

- 1 nur rote Punkte clustern
 - mit Gonzalez
- 2 alle blauen Punkte zu ihren Partnern zuweisen
- 3 Ausreißer bleiben bestehen



Clustern mit Fairlets

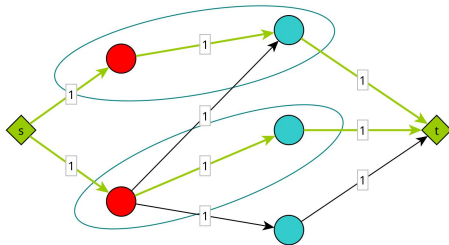
- 1 nur rote Punkte clustern
 - mit Gonzalez
- 2 alle blauen Punkte zu ihren Partnern zuweisen
- 3 Ausreißer bleiben bestehen



Finden der Fairlets - Anchor Variante

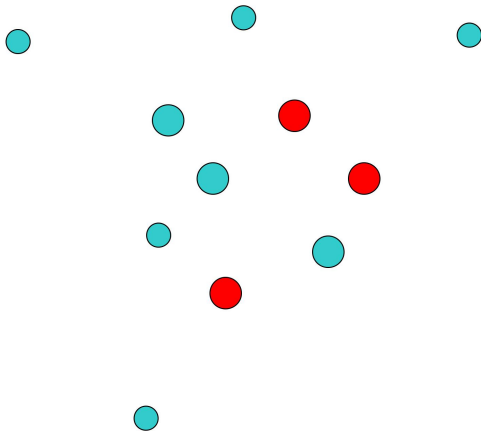
- 1 Anker für jedes Knotenpaar bestimmen
 - $\rightarrow$ max Abstand der Partner zum Anker minimal
- 2 bipartiter Graph mit roten und blauen Knoten aufbauen
- 3 mit Quelle s und Senke t zum Flussnetzwerk machen
- 4 Kanten $s \rightarrow$ Rot und Blau $\rightarrow t$ einfügen
- 5 Kanten zwischen den Mengen nach Bedingung:
 - $r_{test} \geq \text{anchor-dist}(a, b)$ dann Kante
- 6 maximaler Fluss berechnen
- 7 bestes überdeckendes Matching suchen
- 8 gematchte Punkte sind Fairlets
- 9 nicht gematchte Punkte $\rightarrow$ Ausreißer

Beispiel für ein Flussnetzwerk
 $|f| = 2$



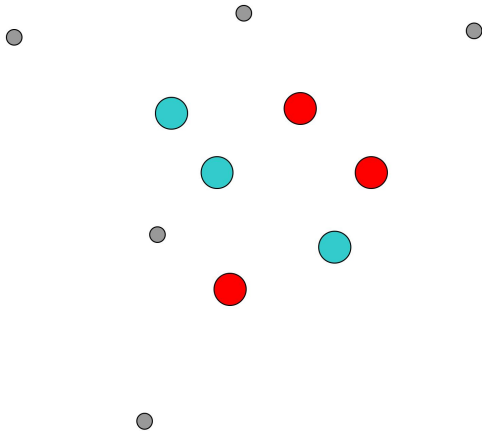
Clustern mit Fairlet-Anker

- 1 alle Punkte ohne Ausreißer clustern
- 2 Zentren und ihre Fairletpartner sich selbst zuweisen
 - → Center-Awareness
- 3 Punkte dem nächsten Zentrum ihres Ankers zuweisen



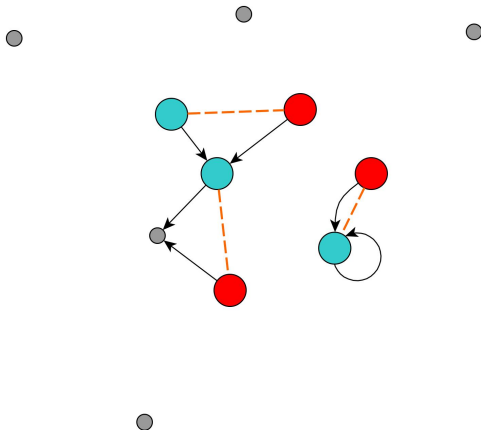
Clustern mit Fairlet-Anker

- 1 alle Punkte ohne Ausreißer clustern
- 2 Zentren und ihre Fairletpartner sich selbst zuweisen
 - → Center-Awareness
- 3 Punkte dem nächsten Zentrum ihres Ankers zuweisen



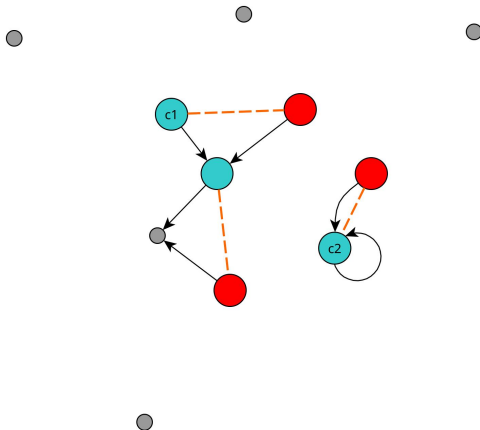
Clustern mit Fairlet-Anker

- 1 alle Punkte ohne Ausreißer clustern
- 2 Zentren und ihre Fairletpartner sich selbst zuweisen
 - → Center-Awareness
- 3 Punkte dem nächsten Zentrum ihres Ankers zuweisen



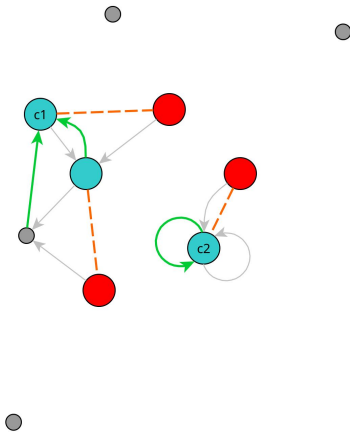
Clustern mit Fairlet-Anker

- 1 alle Punkte ohne Ausreißer clustern
- 2 Zentren und ihre Fairletpartner sich selbst zuweisen
 - → Center-Awareness
- 3 Punkte dem nächsten Zentrum ihres Ankers zuweisen



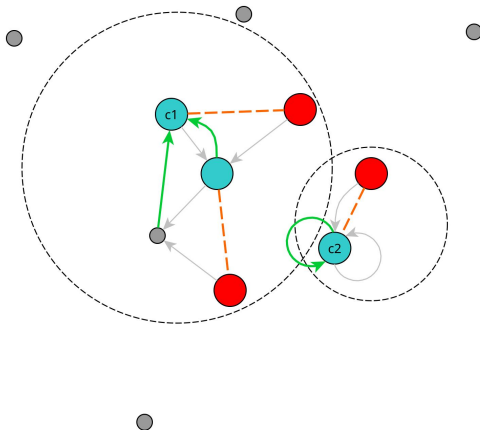
Clustern mit Fairlet-Anker

- 1 alle Punkte ohne Ausreißer clustern
- 2 Zentren und ihre Fairletpartner sich selbst zuweisen
 - → Center-Awareness
- 3 Punkte dem nächsten Zentrum ihres Ankers zuweisen



Clustern mit Fairlet-Anker

- 1 alle Punkte ohne Ausreißer clustern
- 2 Zentren und ihre Fairletpartner sich selbst zuweisen
 - → Center-Awareness
- 3 Punkte dem nächsten Zentrum ihres Ankers zuweisen



- Red-Clustering hat Approximationsgarantie von 4
- Fast-Anchor-Clustering hat Approximationsgarantie von 3
- Laufzeit beider Algorithmen $\in \mathcal{O}(n^3)$

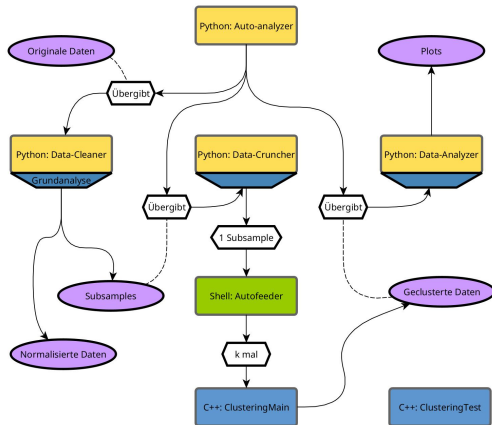
Welche Daten wurden verwendet

Angelehnt an das Paper von Chierichetti et al.¹

Datensatz	Critical-Feature	Verhältnis des Critical-Feature	Verwendete numerische Features	Sub-sample-Größe
Zensusdaten	sex (male = 0; female = 1)	Male 21790 / Female 10771 $\approx$ 2.02	age, fnlwgt, education-num, capital-gain, hours-perweek	600
Bankdaten	bin-marital (selbsterzeugt aus Spalte "marital") (single = 0; divorced = 0; married = 1)	Es gibt single, married und divorced. Zähle single und divorced zusammen. Verhältnis: 27214 married / 17997 not-married $\approx$ 1.51	age, balance, duration of call	1000
Diabetesdaten	sex (male = 0; female = 1)	Verhältnis Male 47055 / Female 54708 $\approx$ 0.86	age, time-in-hospital	1000

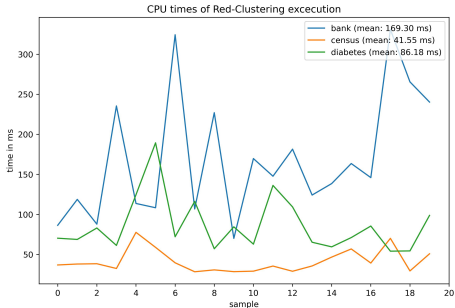
¹Flavio Chierichetti u. a. *Fair Clustering Through Fairlets*. 2018. [arXiv: 1802.05733 \[cs.LG\]](https://arxiv.org/abs/1802.05733).

- Algorithmen in C++
 - effizient
 - Graph-Library (Lemon^a)
- Testen und Datenverwaltung in Python
 - data analysis Library (Pandas)
 - plotting
- Subsamples von Python in C++ Skripte

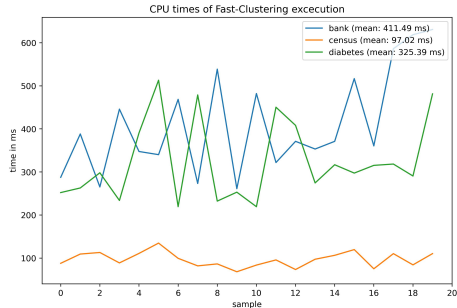


^aA Jüttner, B Dezső und P Kovács. *Lemon Library - open source graph library*. <https://lemon.cs.elte.hu/trac/lemon>.

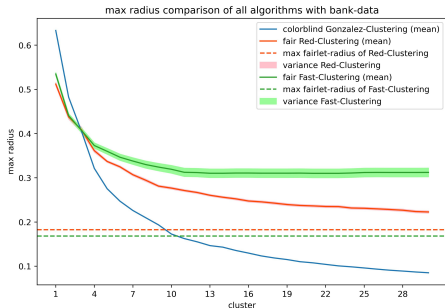
Laufzeit Red-Clustering



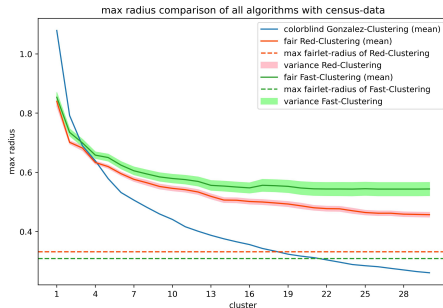
Laufzeit Fast-Anchor-Clustering



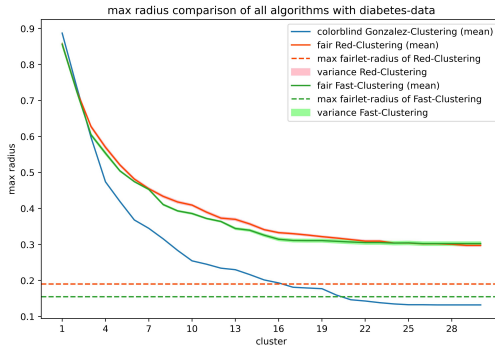
Bankdaten



Zensusdaten

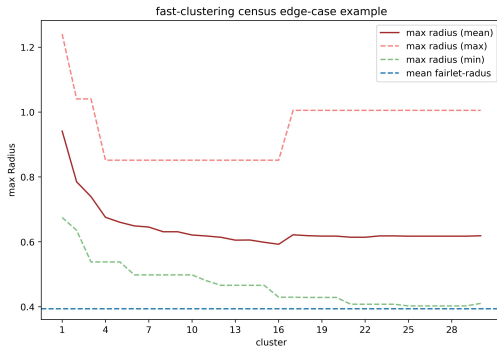


Diabetesdaten



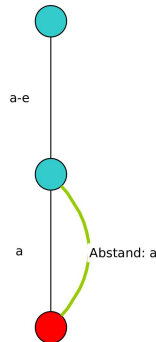
- Überraschend: Fast-Anchor-Clustering schlechter?
- Abstand der Lösungen korreliert mit dimensionalität der Daten
- zusätzlich noch langsamer

Fast-Anchor-Clustering mit Max-Linie

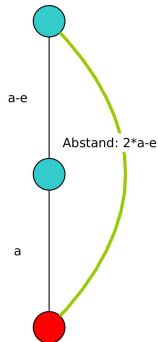


- Für Fairlet-Radius nur Ankerabstand relevant
- → Unterschwellig größere Fairlets möglich
- → Beide Fairlets haben Kennradius a

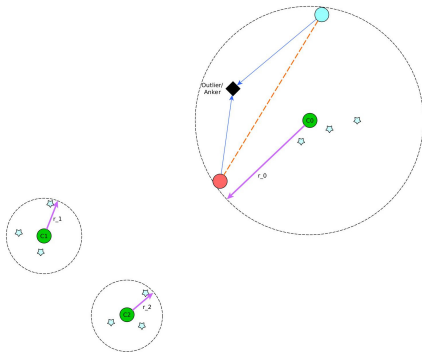
Einzige mögliche Lösung für
Red-Clustering bei $r = a$



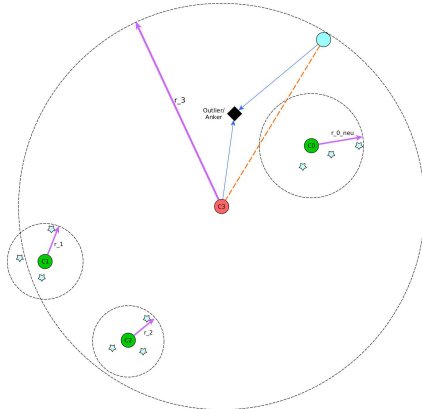
Mögliche Lösung für
Fast-Anchor-Clustering bei $r = a$



Fall $k = 3$



Fall $k = 4$



- Laufzeit
 - redundante Berechnungen reduzieren
 - parallele Berechnungen
- Kosten
 - Gonzalez optimieren
 - nicht Center-Aware arbeiten
- Analyse
 - mehr/diversere Daten
 - optimale Lösungen zum Vergleich

Fast-Anchor-Clustering in einigen Punkten schlechter

- längere Laufzeit
- größere Cluster
- größere Varianz
- höhere Komplexität

-  Chierichetti, Flavio u. a. *Fair Clustering Through Fairlets*. 2018. [arXiv: 1802.05733](#) [cs.LG].
-  Fast, Irina. „Fair k-Center Clustering mit Ausreißern“. Düsseldorf, NRW, Germany, Dez. 2023.
-  Jüttner, A, B Dezső und P Kovács. *Lemon Library - open source graph library*. <https://lemon.cs.elte.hu/trac/lemon>.
-  Schmidt, Melanie. „Kombinatorische Algorithmen für Clusteringprobleme - Vorlesungsskript“. 2023.

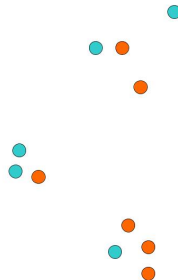
- Approximationsgarantie von 2
- colorblind

Gonzalez Anfang

```

1:  $Z :=$  leere Zentrenmenge
2: Wähle erstes Zentrum  $z_0$  zufällig
3: for all  $x \in P$  do
4:    $d_{min} = \text{dist}(z_0, x)$ 
5: end for
6: for  $i \in 0, \dots, k - 1$  do
7:    $z_i = P(\text{argmax}(d_{min}))$ 
8:   for  $x \in P \setminus Z$  do
9:     Prüfe ob Punkte näher an neuem Zentrum
10:   end for
11: end for
12:  $r_{max} =$  größter Abstand zwischen Punkt und seinem Zentrum
13: return  $(Z, P, r_{max})$ 

```



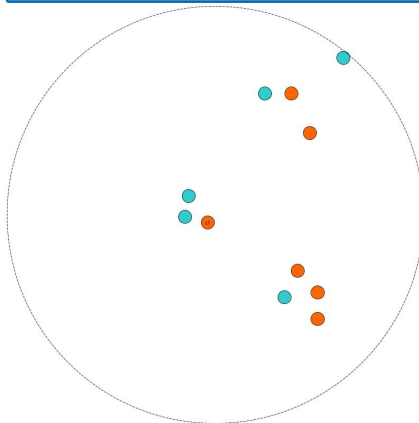
- Approximationsgarantie von 2
- colorblind

```

1:  $Z :=$  leere Zentrenmenge
2: Wähle erstes Zentrum  $z_0$  zufällig
3: for all  $x \in P$  do
4:    $d_{min} = \text{dist}(z_0, x)$ 
5: end for
6: for  $i \in 0, \dots, k - 1$  do
7:    $z_i = P(\text{argmax}(d_{min}))$ 
8:   for  $x \in P \setminus Z$  do
9:     Prüfe ob Punkte näher an neuem Zentrum
10:   end for
11: end for
12:  $r_{max} =$  größter Abstand zwischen Punkt und seinem Zentrum
13: return  $(Z, P, r_{max})$ 

```

Gonzalez: $k=1$



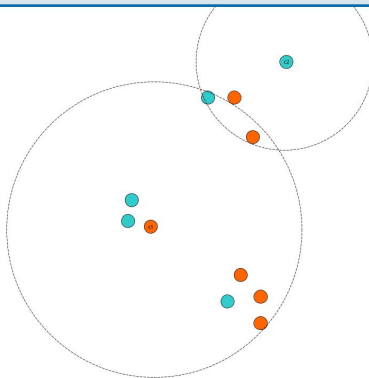
- Approximationsgarantie von 2
- colorblind

```

1:  $Z :=$  leere Zentrenmenge
2: Wähle erstes Zentrum  $z_0$  zufällig
3: for all  $x \in P$  do
4:    $d_{min} = dist(z_0, x)$ 
5: end for
6: for  $i \in 0, \dots, k - 1$  do
7:    $z_i = P(argmax(d_{min}))$ 
8:   for  $x \in P \setminus Z$  do
9:     Prüfe ob Punkte näher an neuem Zentrum
10:   end for
11: end for
12:  $r_{max} =$  größter Abstand zwischen Punkt und seinem Zentrum
13: return  $(Z, P, r_{max})$ 

```

Gonzalez: $k=2$



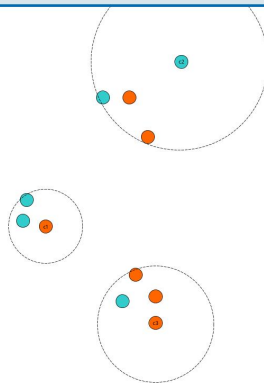
- Approximationsgarantie von 2
- colorblind

```

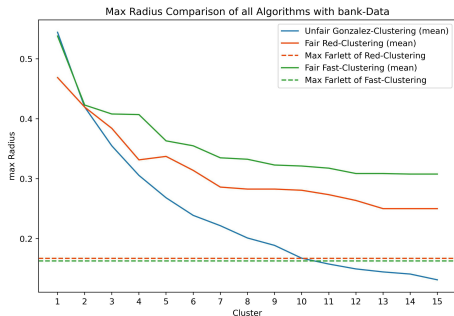
1:  $Z :=$  leere Zentrenmenge
2: Wähle erstes Zentrum  $z_0$  zufällig
3: for all  $x \in P$  do
4:    $d_{min} = dist(z_0, x)$ 
5: end for
6: for  $i \in 0, \dots, k - 1$  do
7:    $z_i = P(argmax(d_{min}))$ 
8:   for  $x \in P \setminus Z$  do
9:     Prüfe ob Punkte näher an neuem Zentrum
10:   end for
11: end for
12:  $r_{max} =$  größter Abstand zwischen Punkt und seinem Zentrum
13: return  $(Z, P, r_{max})$ 

```

Gonzalez: $k=3$



Fast-Anchor with simple max-flow



Fast-Anchor with min-cost-flow

